Homework 2: Route Finding

111550170 卓建均

Part I. Implementation:

Part 1. Breadth-first Search

First, I read graph data from a CSV file using csv.DictReader, storing it as an adjacency list in the edge dictionary. Then initialize a dictionary ‘distance’ to track the shortest distance from start node to each node, and a queue ‘Q’ for BFS traversal. Next, enqueue the start node with distance 0 into Q, then iteratively dequeue nodes, update distances, and enqueue adjacent unvisited nodes until the end node is reached or Q is empty. Once the traversal is complete, reconstruct the shortest path from the start node to the end node using pointers stored in the distance dictionary. Finally, return the reconstructed path, the distance of the shortest path, and the number of visited nodes.

```
edge = {}
with open(edgeFile, newline= '') as csvfile:
    rows = csv.DictReader(csvfile)
    for row in rows:
        s = int(row['start'])
        e = int(row['end'])
        d = float(row['distance'])
        if s in edge:
            edge[s].append((e, d))
        else:
            edge[s] = [(e, d)]
distance = {start: (0, None)}
Q = Queue()
Q.put_nowait((start, 0))
while not Q.empty():
    cur_node, cur_dist = Q.get_nowait()
    if cur_node not in edge:
        continue
    for next_node, d in edge[cur_node]:
        if next_node in distance:
            continue
        distance[next_node] = (cur_dist + d, cur_node)
        Q.put_nowait((next_node, cur_dist + d))
        if next_node == end:
            break
path = [end]
while distance[path[-1]][1] is not None:
    path.append(distance[path[-1]][1])
path.reverse()
dist = distance[end][0]
num_visited = len(distance)
return path, dist, num_visited
```

Part 2. Depth-first Search

Same as BFS, I read the graph data from a CSV file using csv.DictReader, storing it as an adjacency list in the edge dictionary. Then I implement DFS using a stack, I initialize a stack with the start node and its distance 0, and a dictionary 'distance' to track the shortest distance from the start node to each node. Next, I start DFS traversal from the start node. It iteratively pops nodes from the stack, explores their adjacent nodes, updates distances, and pushes unvisited adjacent nodes onto the stack until the end node is reached or the stack becomes empty. After that, I reconstruct the path from the end node to the start node using pointers stored in the distance dictionary then reverse it, and return the reconstructed path, the distance of the path, and the number of visited nodes.

```
edge = {}
with open(edgeFile, newline='') as csvfile:
    rows = csv.DictReader(csvfile)
    for row in rows:
        s = int(row['start'])
        e = int(row['end'])
        d = float(row['distance'])
        if s in edge:
            edge[s].append((e, d))
        else:
            edge[s] = [(e, d)]
stack = [(start, 0)]
distance = {start: (0, None)}
while stack:
    cur_node, cur_dist = stack.pop()
    if cur_node not in edge:
        continue
    for next_node, d in edge[cur_node]:
        if next_node in distance:
            continue
        distance[next_node] = (cur_dist + d, cur_node)
        stack.append((next_node, cur_dist + d))
        if next_node == end:
            break
path = [end]
while distance[path[-1]][1] is not None:
    path.append(distance[path[-1]][1])
path.reverse()
dist = distance[end][0]
num_visited = len(distance)
return path, dist, num_visited
```

Part 3. Uniform Cost Search

Implementing UCS algorithm, I read the graph data from a CSV file using csv.DictReader, storing it as an adjacency list in the edge dictionary. Then I initialize a dictionary ‘distance’ to track the shortest distance from the start node to each node, and a priority queue ‘pq’ to store nodes based on their priority (current distance from the start node). Next, repeatedly dequeue nodes from the priority queue until it finds the target node or the priority queue becomes empty. For each dequeued node, we explore its adjacent nodes. If a shorter path to an adjacent node is found, update its distance in the dictionary ‘distance’ and enqueue the adjacent node along with its updated distance into the priority queue. Once the target node is reached, reconstruct the shortest path from the end node to the start node using pointers stored in the distance dictionary. and return the reconstructed path in reverse order, along with the distance of the shortest path and the number of visited nodes.

```
edge = {}
with open(edgeFile, newline='') as csvfile:
    rows = csv.DictReader(csvfile)
    for row in rows:
        s = int(row['start'])
        e = int(row['end'])
        d = float(row['distance'])
        if s in edge:
            edge[s] += [(e, d)]
        else:
            edge[s] = [(e, d)]
distance = {start: (0, None)}
pq = PriorityQueue()
pq.put((0, start))
while not pq.empty():
    cur_dist, cur_node = pq.get()
    if cur_node == end:
        break
    if cur_node not in edge:
        continue
    for next_node, next_dist in edge[cur_node]:
        new_dist = cur_dist + next_dist
        if next_node not in distance or new_dist < distance[next_node][0]:
            distance[next_node] = (new_dist, cur_node)
            pq.put((new_dist, next_node))
path = [end]
while path[-1] != start:
    path.append(distance[path[-1]][1])
path.reverse()
dist = distance[end][0]
num_visited = len(distance)
return path, dist, num_visited
```

Part 4. A* Search

First, I convert the start and end nodes to strings and initialize dictionaries to store graph edges and heuristic values. It reads edge and heuristic data from CSV files. The edge data represents the graph's connectivity, while the heuristic data provides estimated distances from each node to the goal node. Then, I initialize a priority queue ‘pq’ to store nodes based on their priority, which is determined by the sum of the current distance and the heuristic estimate to the goal node. The start node is added to the priority queue with a priority of 0. The main loop dequeues nodes from the priority queue until it reaches the end node or the priority queue becomes empty. For each dequeued node, it explores its adjacent nodes. If a shorter path to an adjacent node is found, I update the distance in the distance dictionary and enqueue the adjacent node with its updated priority into the priority queue. After finding the shortest path or exploring all nodes, I reconstruct the shortest path from the end node to the start node using pointers stored in the distance dictionary and then reverse it to ensure the correct order. Finally, it returns the shortest path, the distance of the shortest path, and the number of visited nodes.

```
start = str(start)
end = str(end)
edge = {}
with open(edgeFile, newline='') as csvfile:
    rows = csv.reader(csvfile)
    next(rows)
    for row in rows:
        s = row[0]
        e = row[1]
        d = float(row[2])
        h = float(row[3])
        if s in edge:
            edge[s][e] = (d, h)
        else:
            edge[s] = {e: (d, h)}
heuristic = {}
with open(heuristicFile, newline='') as csvfile:
    rows = csv.reader(csvfile)
    title = next(rows)
    for row in rows:
        node = row[0]
        heuristic[node] = {title[i]: float(row[i]) for i in range(1, len(row))}
pq = queue.PriorityQueue()
pq.put((heuristic[start][end], start, -1, 0))
From = {}
distance = {}
path = []
dist = 0
num_visited = 0
while not pq.empty():
    now = pq.get()
    if now[1] in From:
        continue
```

```
    distance[now[1]] = now[0]
    From[now[1]] = now[2]
    num_visited += 1
    if now[1] == end:
        break
    if now[1] not in edge:
        continue
    for i in edge[now[1]]:
        temp = now[3] + edge[now[1]][i][0] + edge[now[1]][i][1]
        if i not in distance or temp < distance[i]:
            pq.put((temp, i, now[1], now[3] + edge[now[1]][i][0]))
            distance[i] = temp
now = end
while From[now] != -1:
    dist += edge[From[now]][now][0]
    path.append(int(now))
    now = From[now]
path.append(int(now))
path.reverse()
return path, dist, num_visited
```

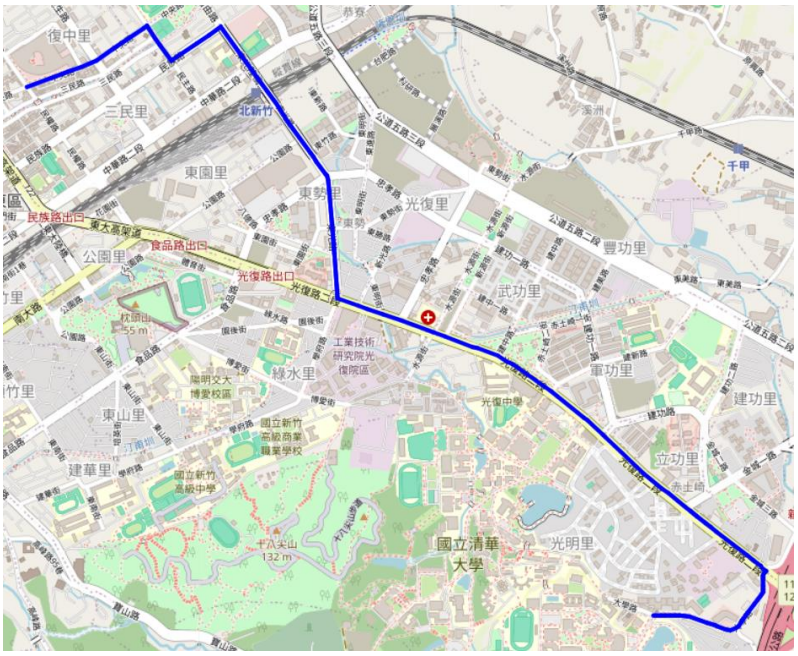
Part II. Results & Analysis:

Test 1.

From National Yang Ming Chiao Tung University (ID: 2270143902) to Big City Shopping Mall (ID: 1079387396)

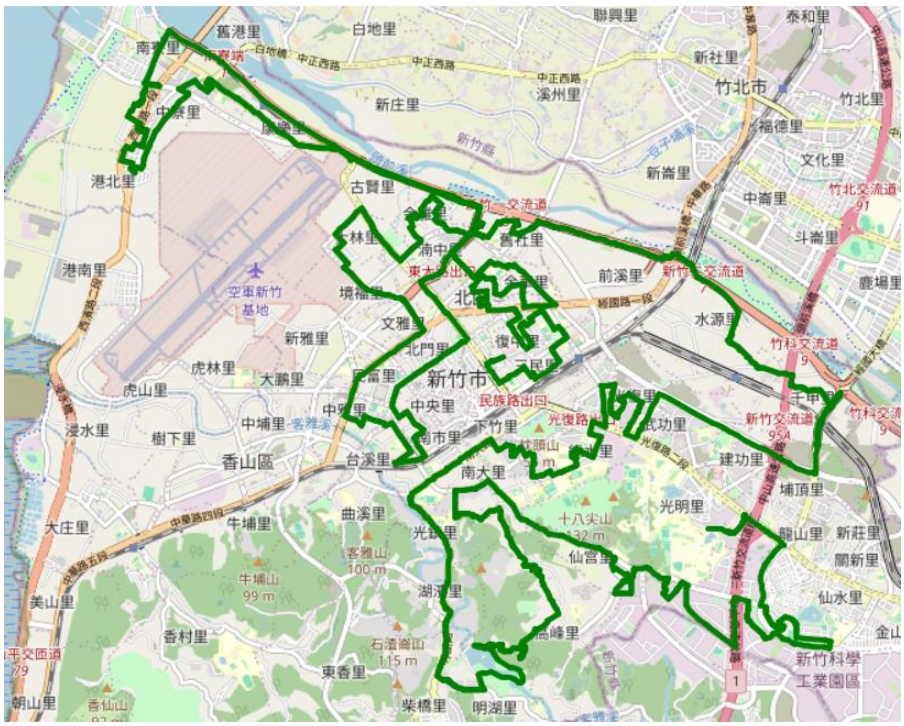
BFS:

```
The number of nodes in the path found by BFS: 88
Total distance of path found by BFS: 4978.8820000000005 m
The number of visited nodes in BFS: 12054
```



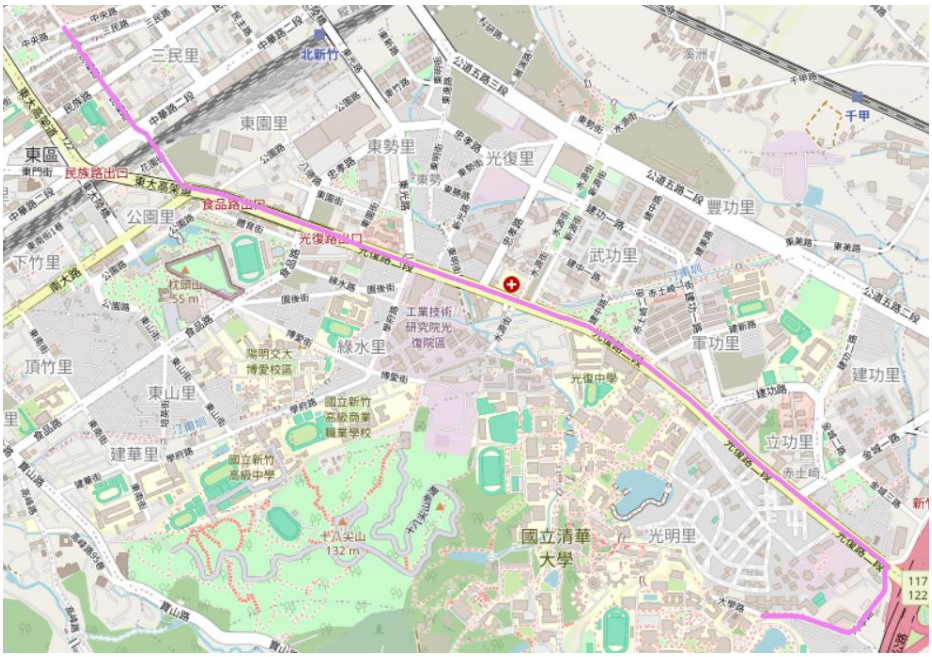
DFS(stack):

```
The number of nodes in the path found by DFS: 1718
Total distance of path found by DFS: 75504.31499999983 m
The number of visited nodes in DFS: 12054
```



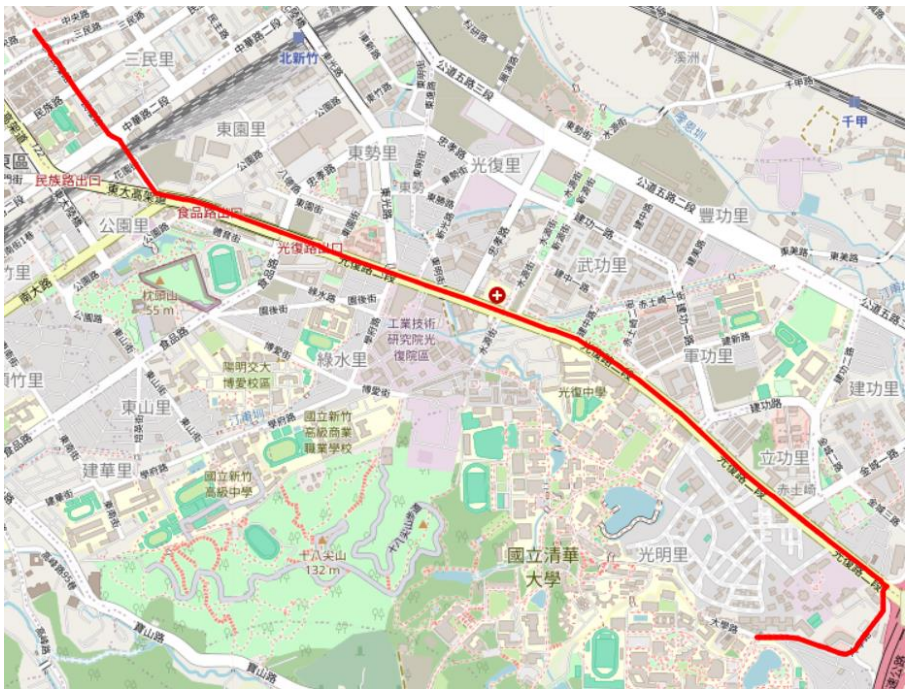
UCS:

```
The number of nodes in the path found by UCS: 89
Total distance of path found by UCS: 4367.881 m
The number of visited nodes in UCS: 5158
```



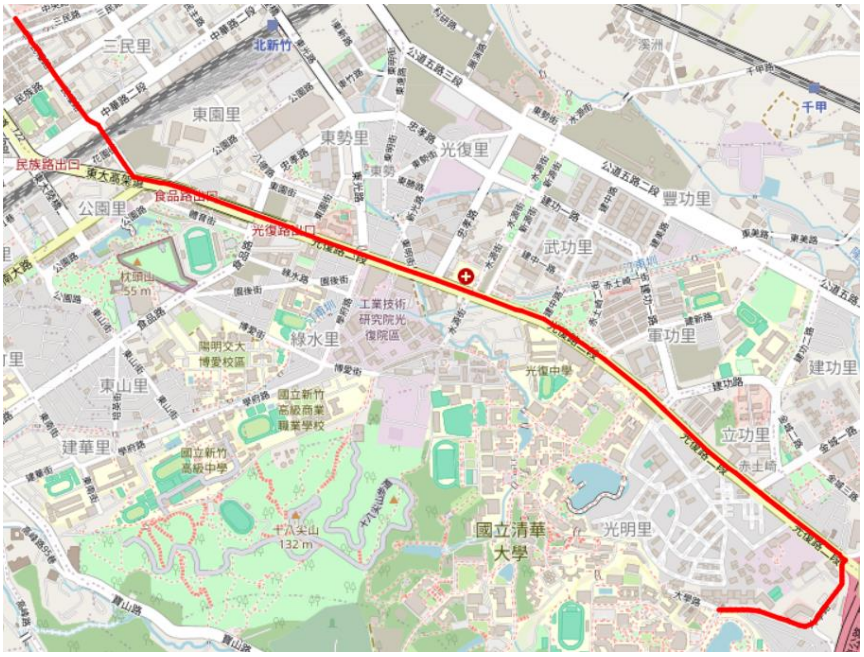
A*:

```
The number of nodes in the path found by A* search: 89
Total distance of path found by A* search: 4367.8809999999985 m
The number of visited nodes in A* search: 5065
```



Part 6. A*(time):

```
The number of nodes in the path found by A* search: 89
Total second of path found by A* search: 320.87823163083146 s
The number of visited nodes in A* search: 1934
```



Test 2.

From Hsinchu Zoo (ID: 426882161) to COSTCO Hsinchu Store (ID: 1737223506)

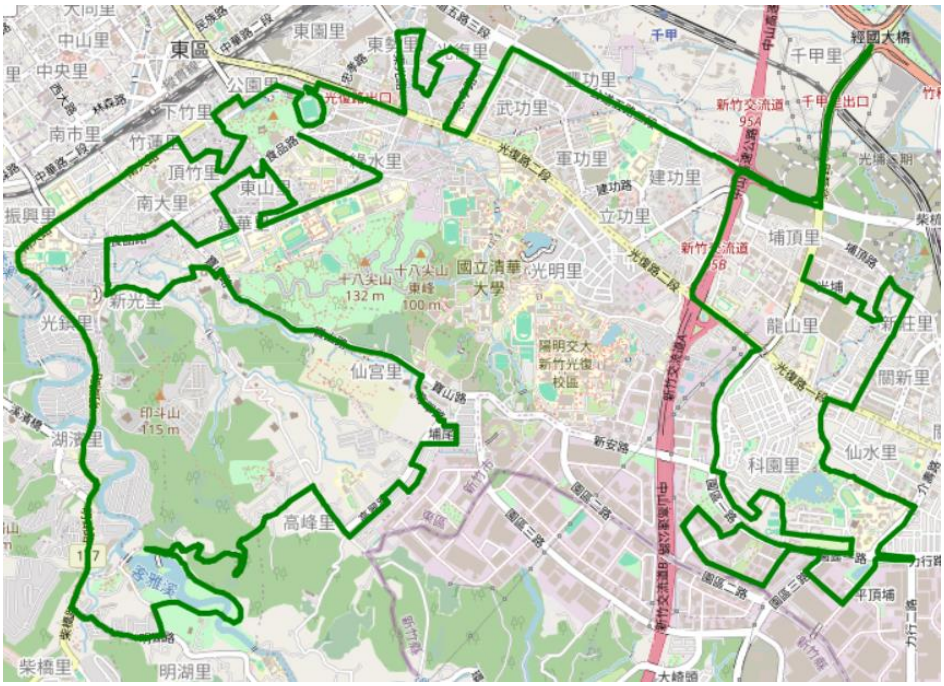
BFS:

```
The number of nodes in the path found by BFS: 60
Total distance of path found by BFS: 4215.521 m
The number of visited nodes in BFS: 12054
```



DFS(stack):

```
The number of nodes in the path found by DFS: 930
Total distance of path found by DFS: 38752.30799999996 m
The number of visited nodes in DFS: 12054
```



UCS:

```
The number of nodes in the path found by UCS: 63
Total distance of path found by UCS: 4101.84 m
The number of visited nodes in UCS: 7318
```



A*:

```
The number of nodes in the path found by A* search: 63
Total distance of path found by A* search: 4101.84 m
The number of visited nodes in A* search: 7203
```



Part 6. A*(time):

```
The number of nodes in the path found by A* search: 63
Total second of path found by A* search: 304.4436634360301 s
The number of visited nodes in A* search: 2870
```



Test 3.

From National Experimental High School At Hsinchu Science Park (ID: 1718165260) to Nanliao Fighing Port (ID: 8513026827)

BFS:

```
The number of nodes in the path found by BFS: 183
Total distance of path found by BFS: 15442.395000000002 m
The number of visited nodes in BFS: 12054
```



DFS(stack):

```
The number of nodes in the path found by DFS: 900
Total distance of path found by DFS: 39219.99299999996 m
The number of visited nodes in DFS: 12054
```



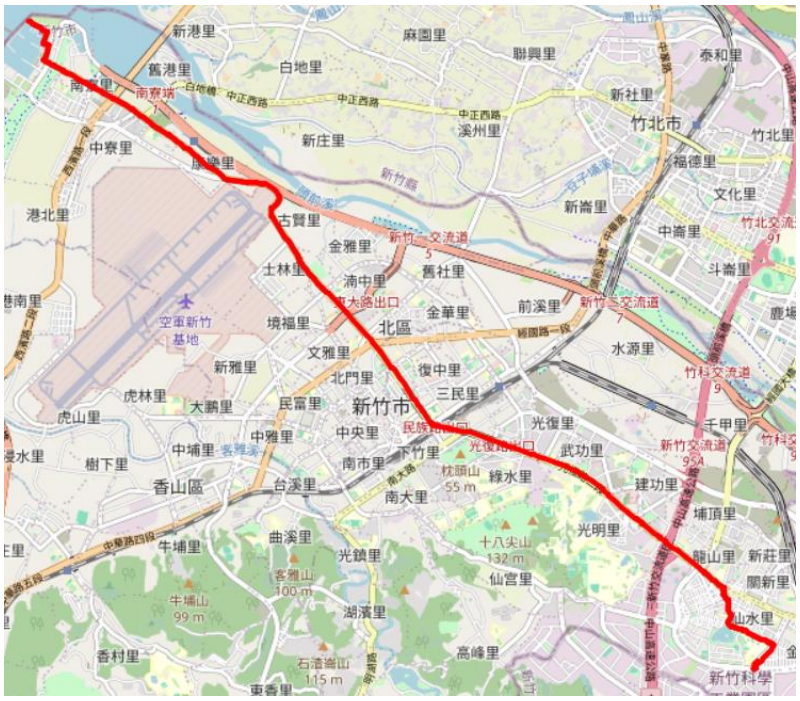
UCS:

```
The number of nodes in the path found by UCS: 288
Total distance of path found by UCS: 14212.412999999997 m
The number of visited nodes in UCS: 11934
```



A*:

```
The number of nodes in the path found by A* search: 288
Total distance of path found by A* search: 14212.413 m
The number of visited nodes in A* search: 11926
```



Part 6. A*(time):

```
The number of nodes in the path found by A* search: 209
Total second of path found by A* search: 779.527922836848 s
The number of visited nodes in A* search: 8458
```



Part 6. Search with a different heuristic (Bonus):

First, I read edge information from a file, where each edge contains data such as start node, end node, distance, and speed limit. It calculates the time required to traverse each edge based on the given speed limit. Then, I read heuristic information from another file, providing an estimated cost from each node to the end node, considering the maximum speed limit. Using this data, and applied A* algorithm. I maintain a priority queue (heap) to explore nodes based on their estimated total cost, which is the sum of the current time, heuristic cost, and the time to reach the next node. It iteratively explores nodes, updating the shortest path and time to reach each node as it progresses. It stops when the end node is reached or when there are no more nodes to explore. Finally, I reconstruct the shortest path from the start node to the end node and return this path along with the time taken to traverse it and the number of nodes visited during the process.

```
edges = {}
max_speed = 0
with open(edgeFile, newline='') as csvfile:
    rows = csv.DictReader(csvfile)
    for row in rows:
        s = int(row['start'])
        e = int(row['end'])
        d = float(row['distance'])
        t = d / (float(row['speed limit']) / 3.6)
        if s in edges:
            edges[s] += [(e, d, t)]
        else:
            edges[s] = [(e, d, t)]
        if float(row['speed limit']) > max_speed:
            max_speed = float(row['speed limit'])
heuristic = {}
with open(heuristicFile, newline='') as csvfile:
    rows = csv.DictReader(csvfile)
    for row in rows:
        heuristic[int(row['node'])] = float(row[str(end)]) / (max_speed / 3.6)
times = {}
Q = []
heapq.heappush(Q, (0 + heuristic[start], start, None))
while len(Q) > 0:
    cur_time, cur_node, prev_node = heapq.heappop(Q)
    if cur_node in times:
        continue
    times[cur_node] = (cur_time, prev_node)
    if cur_node == end:
        break
    if not cur_node in edges:
        continue
```

```
    for nxt_node, d, t in edges[cur_node]:
        if nxt_node in times:
            continue
        heapq.heappush(Q, (cur_time + t - heuristic[cur_node] + heuristic[nxt_node], nxt_node, cur_node))
path = [end]
while times[path[-1]][1] != None:
    path.append(times[path[-1]][1])
path.reverse()
time = times[end][0]
num_visited = len(times)
return path, time, num_visited
```

Part III. Question Answering:

1. Please describe a problem you encountered and how you solved it.

A: The first problem I encountered was the difficulties executing Jupyter Notebook due to inadequacies in my computer setup. I don't know how to run .ipynb file on VS Code, so it takes me some time to learn how to resolve it by watching for videos on youtube. The second problem I'm facing revolves around understanding and implementing two algorithms: UCS and A* algorithm. Both of these algorithms are new concepts to me, so I've spent considerable time researching and trying to comprehend their inner workings. I explored the role and significance of the heuristic function in A* algorithm. I finally came out with the function that works.

2. Besides speed limit and distance, could you please come up with another attribute that is essential for route finding in the real world? Please explain the rationale.

A: Besides speed limit and distance, I think that real-world route finding should also consider traffic congestion. Heavy traffic can significantly increase travel time, making it essential to factor in congestion levels when selecting the optimal route. Furthermore, traffic accidents can cause unexpected delays, requiring route finding algorithms to adapt dynamically to changing conditions. Therefore, incorporating real-time traffic data, including congestion and accident information, is crucial for efficient navigation in the real world.

3. As mentioned in the introduction, a navigation system involves mapping, localization, and route finding. Please suggest possible solutions for mapping and localization components?

A: For mapping, satellite imagery and data from platforms like Google Maps can be utilized to construct a graph representation of roads and intersections. This involves converting the map into a directed weighted graph, where intersections become nodes and roads become edges with weights inversely proportional to the speed limit.

As for localization, GPS systems, commonly integrated into smartphones, can precisely determine the user's current position on the map. This process may require an internet connection for execution.

4. The estimated time of arrival (ETA) is one of the features of Uber Eats. To provide accurate estimates for users, Uber Eats needs to dynamically update ETA based on their mechanism. Please define a dynamic heuristic equation for ETA and explain the rationale of your design. Hint: You can consider meal prep time, delivery priority, multiple orders, etc.

A: In this case, I consider the following factors:

Preparation Time (A): The time required for the restaurant to prepare the order.

Distance to Restaurant (B): The estimated time for the delivery driver to reach the restaurant. This can be calculated based on the straight-line distance between the delivery driver and the restaurant, divided by the average speed of the delivery driver.

Distance to User's Location (C): The estimated time for the delivery driver to reach the user's location. This can be calculated similarly to the distance to the restaurant.

Traffic Conditions (D): The estimated delay caused by traffic conditions.

Considering these factors, the dynamic heuristic equation for ETA could be:

ETA=Preparation Time (A) + max(Distance to Restaurant (B),Distance to User’s Location (C)) + Traffic Conditions (D)

This equation dynamically adjusts the ETA based on whether the delivery driver has already picked up the order or not. If the order has not been picked up yet, I consider the maximum of the time it takes to reach the restaurant and the time it takes to reach the user's location. This ensures that the ETA accounts for the longest potential delay. After the order is picked up, I only consider the time it takes to reach the user's location, as the preparation time at the restaurant has already been factored in.